

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

NOMBRE: SERRANO ABARCA ALONSO MIKKAIL

PARALELO: SOF-S-V-8-4

**PRUEBAS EJECUTADAS AL MÓDULO MÉDICOS DEL SISTEMA DE
GESTIÓN VETERINARIA**

1er Caso de Prueba al método guardarMedico()

```
@Test
public void testGuardarMedico() {

    ModeloMedico medico = new ModeloMedico();

    medico.setNombres("Carlos");
    medico.setApellidos("Perez");
    medico.setEspecialidad("Caninos");
    medico.setTelefono("0999999999");

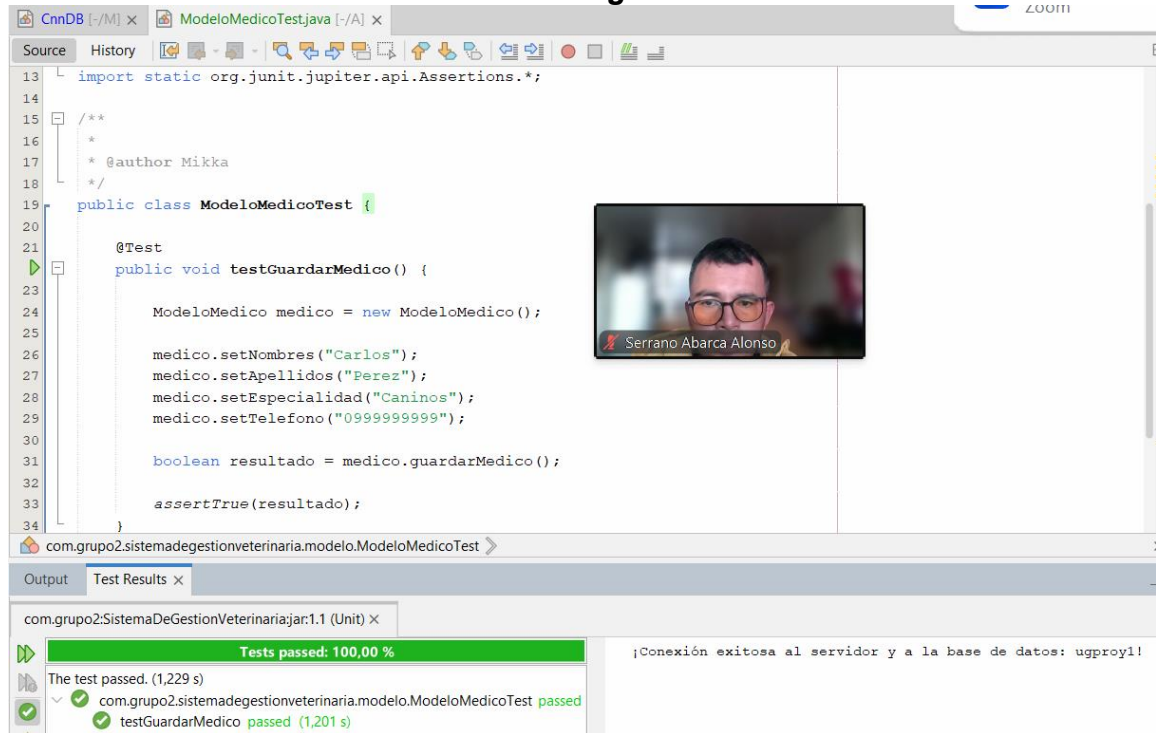
    boolean resultado = medico.guardarMedico();

    assertTrue(resultado);
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Nombres	Carlos	Médico guardado correctamente	Médico guardado correctamente	Aceptable
Apellidos	Pérez	Médico guardado correctamente	Médico guardado correctamente	Aceptable
Especialidad	Caninos	Médico guardado correctamente	Médico guardado correctamente	Aceptable
Teléfono	0999999999	Médico guardado correctamente	Médico guardado correctamente	Aceptable
Método guardarMedico()	válidos completos	true	true	Aceptable

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.



```
13 import static org.junit.jupiter.api.Assertions.*;
14
15 /**
16  *
17  * @author Mikka
18  */
19 public class ModeloMedicoTest {
20
21     @Test
22     public void testGuardarMedico() {
23
24         ModeloMedico medico = new ModeloMedico();
25
26         medico.setNombres("Carlos");
27         medico.setApellidos("Perez");
28         medico.setEspecialidad("Caninos");
29         medico.setTelefono("0999999999");
30
31         boolean resultado = medico.guardarMedico();
32
33         assertTrue(resultado);
34     }
35 }
```

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Output Test Results x

com.grupo2:SistemaDeGestionVeterinariaJar:1.1 (Unit) x

Tests passed: 100,00 %

The test passed. (1,229 s)

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest passed

testGuardarMedico passed (1,201 s)

¡Conexión exitosa al servidor y a la base de datos: ugroy1!

La prueba unitaria realizada al método `guardarMedico()` permitió comprobar que el sistema registra correctamente un médico cuando se ingresan datos válidos. El uso de `assertTrue()` confirmó que el método retornó un valor verdadero (`true`), validando el correcto funcionamiento del proceso de guardado.

2do Caso de Prueba al método `listarMedicos()`

```
@Test
public void testListarMedicos() {

    ModeloMedico medico = new ModeloMedico();

    assertNotNull(medico.listarMedicos());

    assertTrue(
        medico.listarMedicos().size() > 0
    );
}
```

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Método listarMedicos()	Registros existentes en la BD	Lista diferente de null	Lista diferente de null	Aceptable
Cantidad de médicos	Médicos registrados previamente	Cantidad mayor a 0	Cantidad mayor a 0	Aceptable
Resultado del método	Consulta de médicos	Lista de médicos cargada correctamente	Lista de médicos cargada correctamente	Aceptable

```

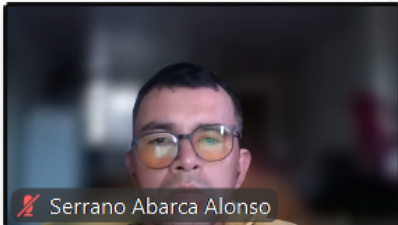
@Test
public void testListarMedicos() {

    ModeloMedico medico = new ModeloMedico();

    assertNotNull(medico.listarMedicos());

    assertTrue(
        medico.listarMedicos().size() > 0
    );
}

```



Serrano Abarca Alonso

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Test Results

com.grupo2:SistemaDeGestionVeterinaria:jar:1.1 (Unit)

Tests passed: 100,00 %

3 tests passed. (1,086 s)

- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest
 - testListarMedicos passed (0,865 s)
 - testGuardarMedico passed (0,193 s)

```

;Conexión exitosa al servidor y
;Conexión exitosa al servidor y
;Conexión exitosa al servidor y

```

La prueba unitaria aplicada al método listarMedicos() permitió comprobar que el sistema recupera correctamente los médicos almacenados en la base de datos. Las validaciones realizadas con assertNotNull() y assertTrue() confirmaron que la lista generada es válida y contiene información registrada en el sistema.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

3er Caso de Prueba al método actualizarMedico()

```
@Test
public void testActualizarMedico() {

    ModeloMedico medico = new ModeloMedico();

    // ID existente en tu BD
    medico.setIdMedico(1);

    medico.setNombres("Carlos Actualizado");
    medico.setApellidos("Perez Actualizado");
    medico.setEspecialidad("Felinos");
    medico.setTelefono("0981111111");

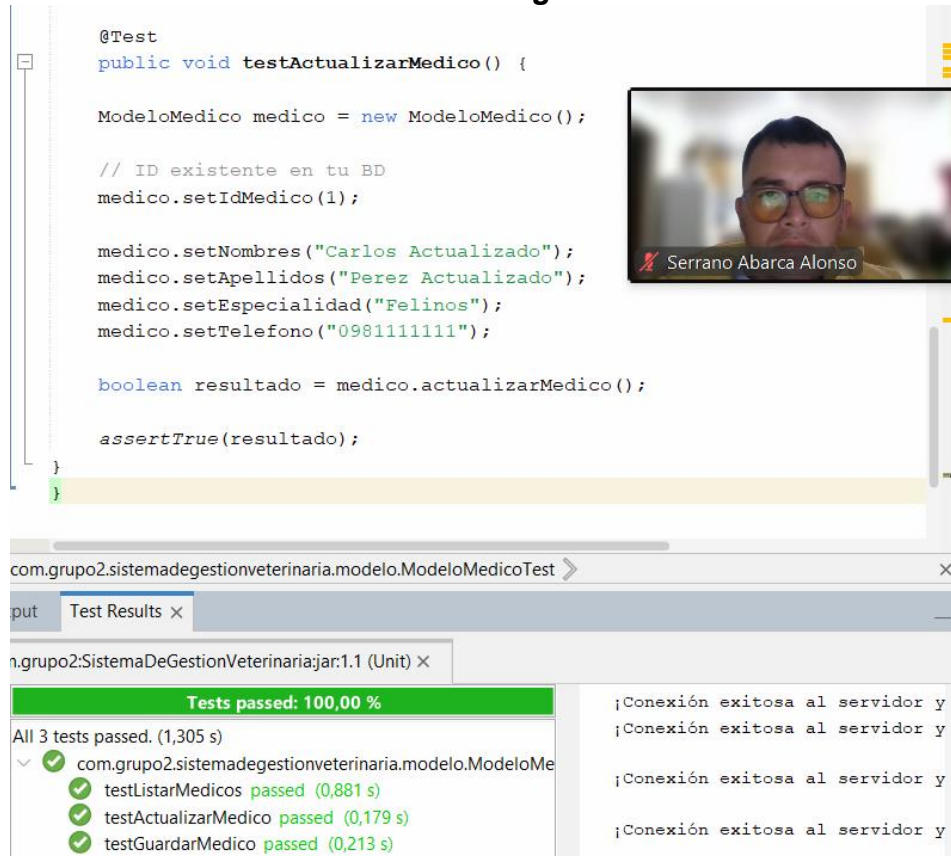
    boolean resultado = medico.actualizarMedico();

    assertTrue(resultado);
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
ID Médico	1	Médico encontrado	Médico encontrado	Aceptable
Nombres	Carlos Actualizado	Datos actualizados	Datos actualizados	Aceptable
Especialidad	Felinos	Especialidad modificada	Especialidad modificada	Aceptable
Teléfono	0981111111	Teléfono actualizado	Teléfono actualizado	Aceptable
Método actualizarMedico()	Datos válidos	true	true	Aceptable

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.



```
@Test
public void testActualizarMedico() {

    ModeloMedico medico = new ModeloMedico();

    // ID existente en tu BD
    medico.setIdMedico(1);

    medico.setNombres("Carlos Actualizado");
    medico.setApellidos("Perez Actualizado");
    medico.setEspecialidad("Felinos");
    medico.setTelefono("0981111111");

    boolean resultado = medico.actualizarMedico();

    assertTrue(resultado);
}
```

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Test Results

n.grupo2:SistemaDeGestionVeterinaria:jar:1.1 (Unit)

Tests passed: 100,00 %

All 3 tests passed. (1,305 s)

- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMe
- testListarMedicos passed (0,881 s)
- testActualizarMedico passed (0,179 s)
- testGuardarMedico passed (0,213 s)

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

La prueba unitaria realizada al método `actualizarMedico()` permitió verificar que el sistema modifica correctamente la información de un médico existente. El uso de `assertTrue()` confirmó que el método retornó un valor verdadero (`true`), validando el correcto funcionamiento del proceso de actualización de datos dentro del sistema veterinario.

4to Caso de Prueba al método `eliminarMedico()`

```
@Test
public void testEliminarMedico() {

    ModeloMedico medico = new ModeloMedico();

    // ID existente para eliminar
    medico.setIdMedico(10);

    boolean resultado = medico.eliminarMedico();

    assertTrue(resultado);
}
```

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
ID Médico	1	Médico encontrado	Médico encontrado	Aceptable
Método eliminarMedico()	ID válido existente	true	true	Aceptable
Eliminación en BD	Registro existente	Registro eliminado	Registro eliminado	Aceptable

```
@Test
public void testEliminarMedico() {

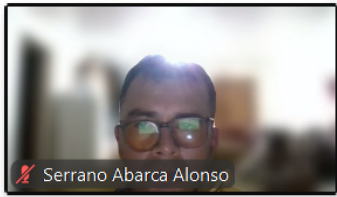
    ModeloMedico medico = new ModeloMedico();

    // ID existente para eliminar
    medico.setIdMedico(10);

    boolean resultado = medico.eliminarMedico();

    assertTrue(resultado);
}

}
```



com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Test Results

m.grupo2:SistemaDeGestionVeterinaria:jar:1.1 (Unit)

Tests passed: 100,00 %

All 3 tests passed. (0,886 s)

- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMe
 - testListarMedicos passed (0,544 s)
 - testEliminarMedico passed (0,188 s)
 - testActualizarMedico passed (0,14 s)

;Conexión exitosa al servidor y
;Conexión exitosa al servidor y
;Conexión exitosa al servidor y
;Conexión exitosa al servidor y

La prueba unitaria aplicada al método `eliminarMedico()` permitió verificar que el sistema elimina correctamente un médico existente en la base de datos. El uso de `assertTrue()` confirmó que el método retornó un valor verdadero (`true`), validando el correcto funcionamiento del proceso de eliminación dentro del sistema veterinario.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

5to Caso de Prueba al método buscarMedico()

```
@Test
public void testBuscarMedico() {

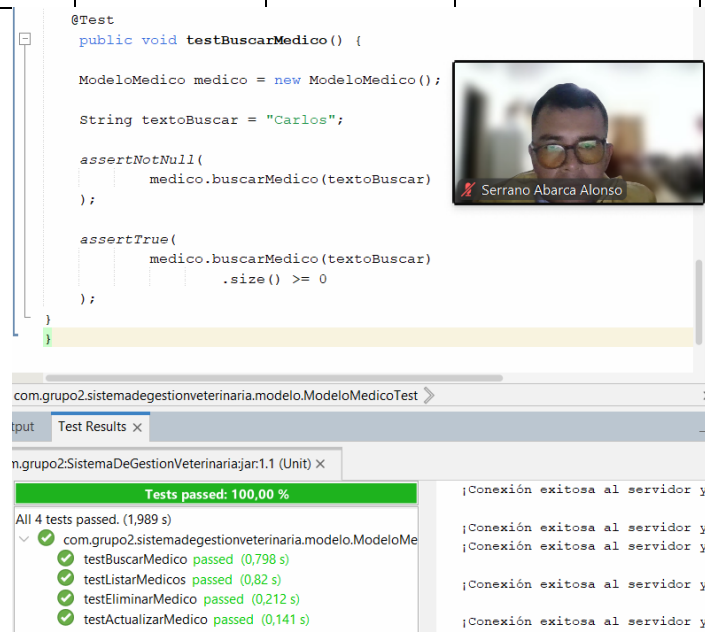
    ModeloMedico medico = new ModeloMedico();

    String textoBuscar = "Carlos";

    assertNotNull(
        medico.buscarMedico(textoBuscar)
    );

    assertTrue(
        medico.buscarMedico(textoBuscar)
            .size() >= 0
    );
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Texto búsqueda	Carlos	Lista diferente de null	Lista diferente de null	Aceptable
Método buscarMedico()	Nombre existente/parcial	Lista de coincidencias	Lista de coincidencias	Aceptable
Resultado búsqueda	"Carlos"	Tamaño mayor o igual a 0	Tamaño mayor o igual a 0	Aceptable



com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Test Results x

n.grupo2.SistemaDeGestionVeterinaria:1.1 (Unit) x

Tests passed: 100,00 %

All 4 tests passed. (1,989 s)

- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest
 - testBuscarMedico passed (0,798 s)
 - testListarMedicos passed (0,82 s)
 - testEliminarMedico passed (0,212 s)
 - testActualizarMedico passed (0,141 s)

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

¡Conexión exitosa al servidor y

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

La prueba unitaria aplicada al método buscarMedico() permitió verificar que el sistema realiza correctamente la búsqueda de médicos utilizando un criterio específico. Las validaciones realizadas con assertNotNull() y assertTrue() confirmaron que el método retorna una lista válida y que el proceso de búsqueda funciona correctamente dentro del sistema veterinario.

Pruebas fallidas

6to Caso de Prueba cuando el teléfono excede más de 10 dígitos

```
@Test
public void testTelefonoExcedeLimite() {

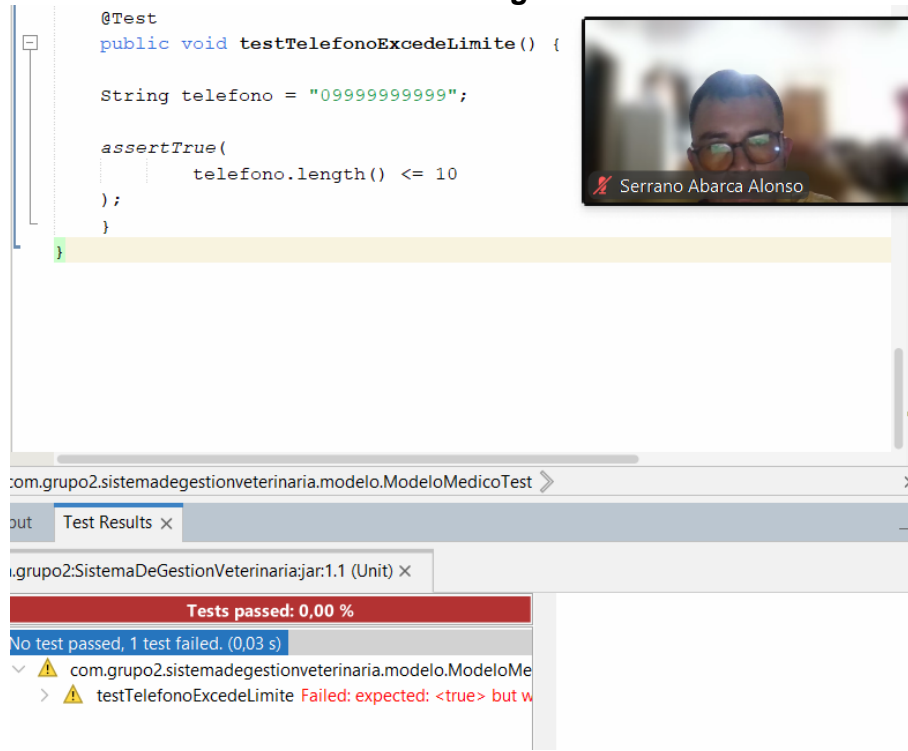
    String telefono = "099999999999";

    assertTrue(
        telefono.length() <= 10
    );
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Teléfono	099999999999	Longitud menor o igual a 10	Longitud igual a 11	Fallo
Validación teléfono	Más de 10 dígitos	Más de 10 dígitos	Validación incorrecta	Fallo

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.



```
@Test
public void testTelefonoExcedeLimite() {

    String telefono = "09999999999";

    assertTrue(
        telefono.length() <= 10
    );
}
```

Test Results x

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Tests passed: 0,00 %

No test passed, 1 test failed. (0,03 s)

com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

testTelefonoExcedeLimite Failed: expected: <true> but was: <false>

La prueba unitaria produjo un resultado fallido (Failed), debido a que el número telefónico excede el límite permitido de caracteres.

La prueba unitaria permitió comprobar que el sistema identifica correctamente números telefónicos inválidos cuando exceden los 10 dígitos permitidos. El resultado fallido confirma el correcto funcionamiento de la validación implementada.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

7mo Caso de prueba cuando se ingresa letras en el campo de teléfono

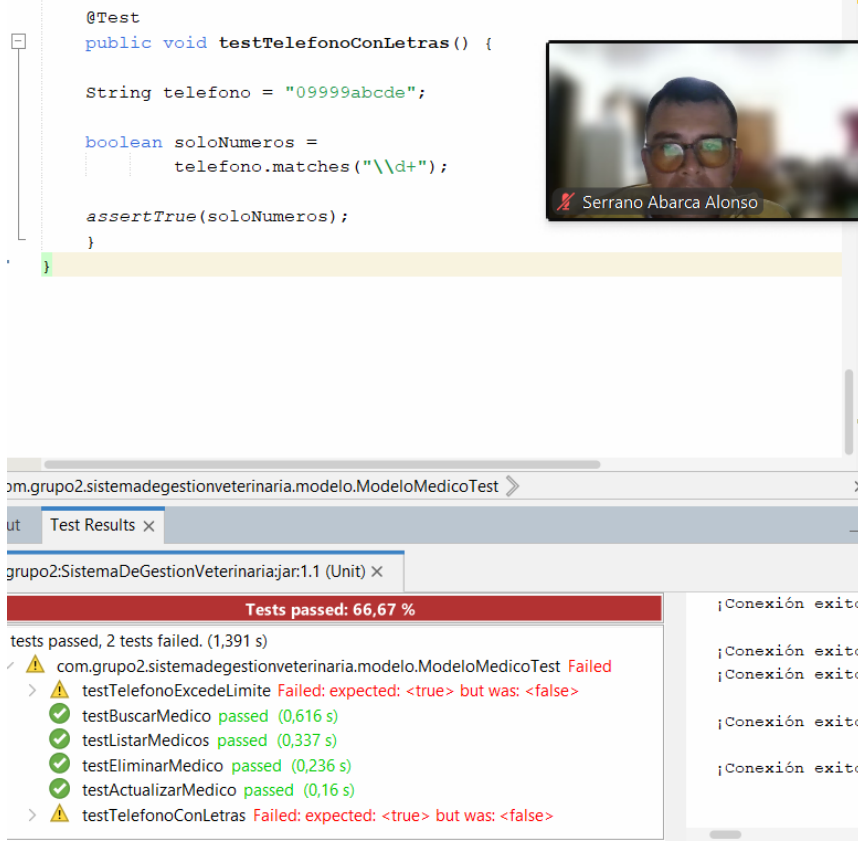
```
@Test
public void testTelefonoConLetras() {

    String telefono = "09999abcde";

    boolean soloNumeros =
        telefono.matches("\\d+");

    assertTrue(soloNumeros);
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Teléfono	09999abcde	Solo números permitidos	No permite ingresar letras	Fallo
Validación caracteres	Datos alfanuméricos	true (solo números)	false	Fallo



The screenshot shows an IDE with a Java test method `testTelefonoConLetras` that is failing. The test expects the phone number `"09999abcde"` to match the regular expression `"\\d+"`, but it fails because it contains letters. A video call window is overlaid on the right side of the IDE, showing a man with glasses and the name `Serrano Abarca Alonso`.

Test Results:

- Tests passed: 66,67 %
- tests passed, 2 tests failed. (1,391 s)
- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest Failed
 - testTelefonoExcedeLimite Failed: expected: <true> but was: <false>
 - testBuscarMedico passed (0,616 s)
 - testListarMedicos passed (0,337 s)
 - testEliminarMedico passed (0,236 s)
 - testActualizarMedico passed (0,16 s)
 - testTelefonoConLetras Failed: expected: <true> but was: <false>

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

La prueba falló correctamente porque el número telefónico no cumple con la validación establecida para aceptar únicamente caracteres numéricos.

La prueba unitaria permitió verificar que el sistema identifica correctamente números telefónicos inválidos cuando contienen letras. El resultado fallido confirma el correcto funcionamiento de la validación de caracteres numéricos implementada en el sistema veterinario.

8vo Caso de prueba cuando se ingresa números en el campo de Nombres

```
@Test
public void testNombresConNumeros() {

    String nombre = "Juan123";

    boolean soloLetras =
        nombre.matches("[a-zA-Z ]+");

    assertTrue(soloLetras);
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Nombres	Juan123	Solo letras permitidas	No permite ingresar números	Fallo
Validación nombres	Caracteres alfanuméricos	No permitir ingreso	Ingreso inválido detectado	Fallo

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

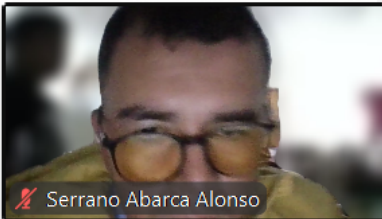
Docente: Ing. Verónica Mendoza Morán MSc.

```
@Test
public void testNombresConNumeros() {

    String nombre = "Juan123";

    boolean soloLetras =
        nombre.matches("[a-zA-Z ]+");

    assertTrue(soloLetras);
}
```



Serrano Abarca Alonso

n.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest

Test Results x

grupo2:SistemaDeGestionVeterinaria:jar:1.1 (Unit) x

Tests passed: 57,14 %

5 tests passed, 5 tests failed. (1,204 s)

- com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest Failed
 - testNombresConNumeros Failed: expected: <true> but was: <false>
 - testTelefonoExcedeLimite Failed: expected: <true> but was: <false>
 - testBuscarMedico passed (0,664 s)
 - testListarMedicos passed (0,295 s)
 - testEliminarMedico passed (0,134 s)
 - testActualizarMedico passed (0,142 s)
 - testTelefonoConLetras Failed: expected: <true> but was: <false>

;Conexión exitosa

;Conexión exitosa

;Conexión exitosa

;Conexión exitosa

;Conexión exitosa

La prueba falló correctamente porque el valor ingresado no cumple con la validación establecida para aceptar únicamente letras y espacios.

La prueba unitaria permitió comprobar que el sistema identifica correctamente nombres inválidos cuando contienen números. El resultado fallido confirma el correcto funcionamiento de la validación de texto implementada en el sistema veterinario.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

9no Caso de prueba cuando se ingresa números en el campo de Apellidos

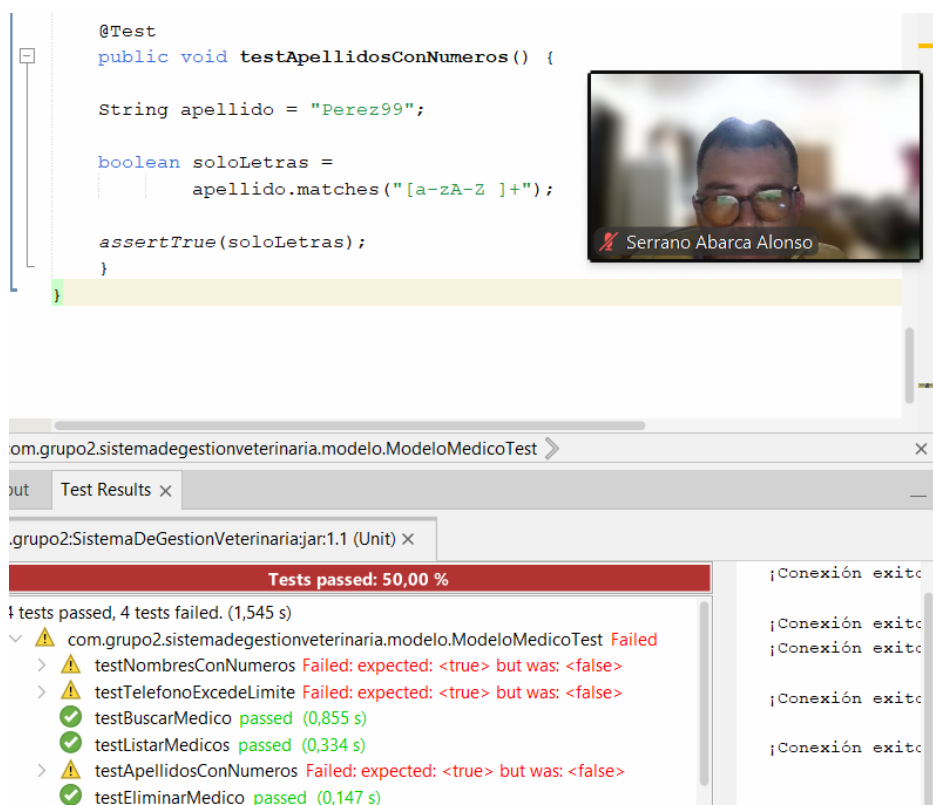
```
@Test
public void testApellidosConNumeros() {

    String apellido = "Perez99";

    boolean soloLetras =
        apellido.matches("[a-zA-Z ]+");

    assertTrue(soloLetras);
}
```

Campos	Datos de entrada	Salida esperada	Salida obtenida	Resultado: Aceptable/Error/Fallo
Apellidos	Perez99	Solo letras permitidas	No permite ingresar números	Fallo
Validación apellidos	Caracteres alfanuméricos	No permitir ingreso	Ingreso inválido detectado	Fallo



The screenshot shows an IDE with a Java test case and its results. The test case is as follows:

```
@Test
public void testApellidosConNumeros() {

    String apellido = "Perez99";

    boolean soloLetras =
        apellido.matches("[a-zA-Z ]+");

    assertTrue(soloLetras);
}
```

The test results window shows the following output:

```
com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest
Test Results
com.grupo2.SistemaDeGestionVeterinaria:jar:1.1 (Unit)
Tests passed: 50,00 %
4 tests passed, 4 tests failed. (1,545 s)
com.grupo2.sistemadegestionveterinaria.modelo.ModeloMedicoTest Failed
  > testNombresConNumeros Failed: expected: <true> but was: <false>
  > testTelefonoExcedeLimite Failed: expected: <true> but was: <false>
  > testBuscarMedico passed (0,855 s)
  > testListarMedicos passed (0,334 s)
  > testApellidosConNumeros Failed: expected: <true> but was: <false>
  > testEliminarMedico passed (0,147 s)
```

The test results window also shows a video feed of a person, identified as Serrano Abarca Alonso.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

La prueba falló correctamente porque el valor ingresado no cumple con la validación establecida para aceptar únicamente letras y espacios.

La prueba unitaria permitió comprobar que el sistema identifica correctamente apellidos inválidos cuando contienen números. El resultado fallido confirma el correcto funcionamiento de la validación de texto implementada en el sistema veterinario.

ANÁLISIS DE LA COMPLEJIDAD CICLOMÁTICA

Método guardarMedico()

```
public void guardarMedico() {  
  
    //-----  
    // VALIDAR CAMPOS VACÍOS  
    //-----  
    if (vista.txtNombres.getText().trim().isEmpty()  
        || vista.txtApellidos.getText().trim().isEmpty()  
        || vista.txtEspecialidad.getText().trim().isEmpty()  
        || vista.txtTelefono.getText().trim().isEmpty()) {  
  
        JOptionPane.showMessageDialog(  
            null,  
            "Todos los campos son obligatorios"  
        );  
  
        return;  
    }  
  
    modelo.setNombres(  
        vista.txtNombres.getText()  
    );  
  
    modelo.setApellidos(  
        vista.txtApellidos.getText()  
    );  
  
    modelo.setEspecialidad(  
        vista.txtEspecialidad.getText()  
    );  
  
    modelo.setTelefono(  
        vista.txtTelefono.getText()  
    );  
  
    modelo.setEstado(  

```

Diagrama de flujo de la complejidad ciclomática:

- 1. Inicio del método `guardarMedico()`.
- 2. Condición de validación de campos vacíos. Si se cumple, se muestra un mensaje de error y se retorna.
- 3. Asignación de valores a los atributos del modelo.

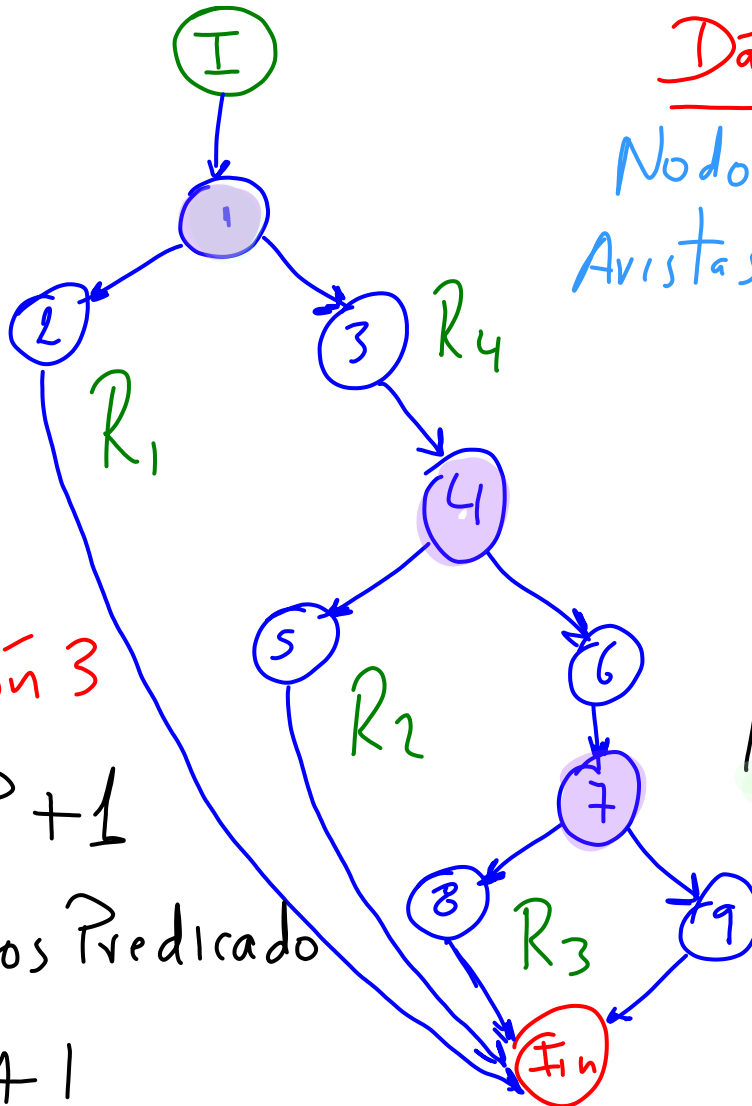
UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

```
vista.chkEstado.isSelected()  
);  
  
4 if (vista.txtTelefono.getText().length() != 10) {  
    JOptionPane.showMessageDialog(  
        null,  
        "El teléfono debe tener 10 dígitos"  
    );  
    return; 5 Fin  
}  
  
boolean resultado =  
    modelo.guardarMedico(); 6  
  
7 if (resultado) {  
    JOptionPane.showMessageDialog(  
        null,  
        "Médico guardado correctamente"  
    );  
    limpiarCampos();  
    listarMedicos();  
} else {  
    JOptionPane.showMessageDialog(  
        null,  
        "Error al guardar médico"  
    );  
} 9  
} 8 Fin
```


UNIVERSIDAD DE GUAYAQUIL
 FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
 CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.



Datos

Nodos = 11 $\Rightarrow N$
 Avistas = 13 $\Rightarrow E$

Definición 1

$$M = E - N + 2P$$

$$M = 13 - 11 + 2(1)$$

$$M = 4$$

Definición 3

$$M = NP + 1$$

NP = nodos Predicado

$$M = 3 + 1$$

$$M = 4$$

Definición 2

$$M = \# R$$

$$M = 4$$

Conclusión:

El método tiene una complejidad ciclomática de 4 ($M = 4$):

- Es mantenible
- Es fácil de probar
- Tiene una estructura lógica clara.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

Método eliminarMedico()

```
//-----  
public void eliminarMedico() {  
  
    //-----  
    // VALIDAR SELECCIÓN  
    //-----  
    if (idMedicoSeleccionado == 0) {  
  
        JOptionPane.showMessageDialog(  
            null,  
            "Seleccione un médico"  
        );  
  
        return;  
    }  
}
```

```
//-----  
// CONFIRMACIÓN  
//-----  
int opcion =  
    JOptionPane.showConfirmDialog(  
        null,  
        "¿Desea eliminar este médico?",  
        "Confirmar eliminación",  
        JOptionPane.YES_NO_OPTION  
    );  
  
if (opcion == JOptionPane.YES_OPTION) {  
  
    modelo.setIdMedico(  
        idMedicoSeleccionado  
    );  
  
    boolean resultado =  
        modelo.eliminarMedico();  
  
    if (resultado) {  
  
    }  
}
```

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

```
JOptionPane.showMessageDialog(  
    null,  
    "Médico eliminado correctamente"  
);  
  
limpiarCampos();  
  
listarMedicos();  
  
idMedicoSeleccionado = 0;  
  
} else {  
    JOptionPane.showMessageDialog(  
        null,  
        "Error al eliminar médico"  
    );  
}  
}
```

7

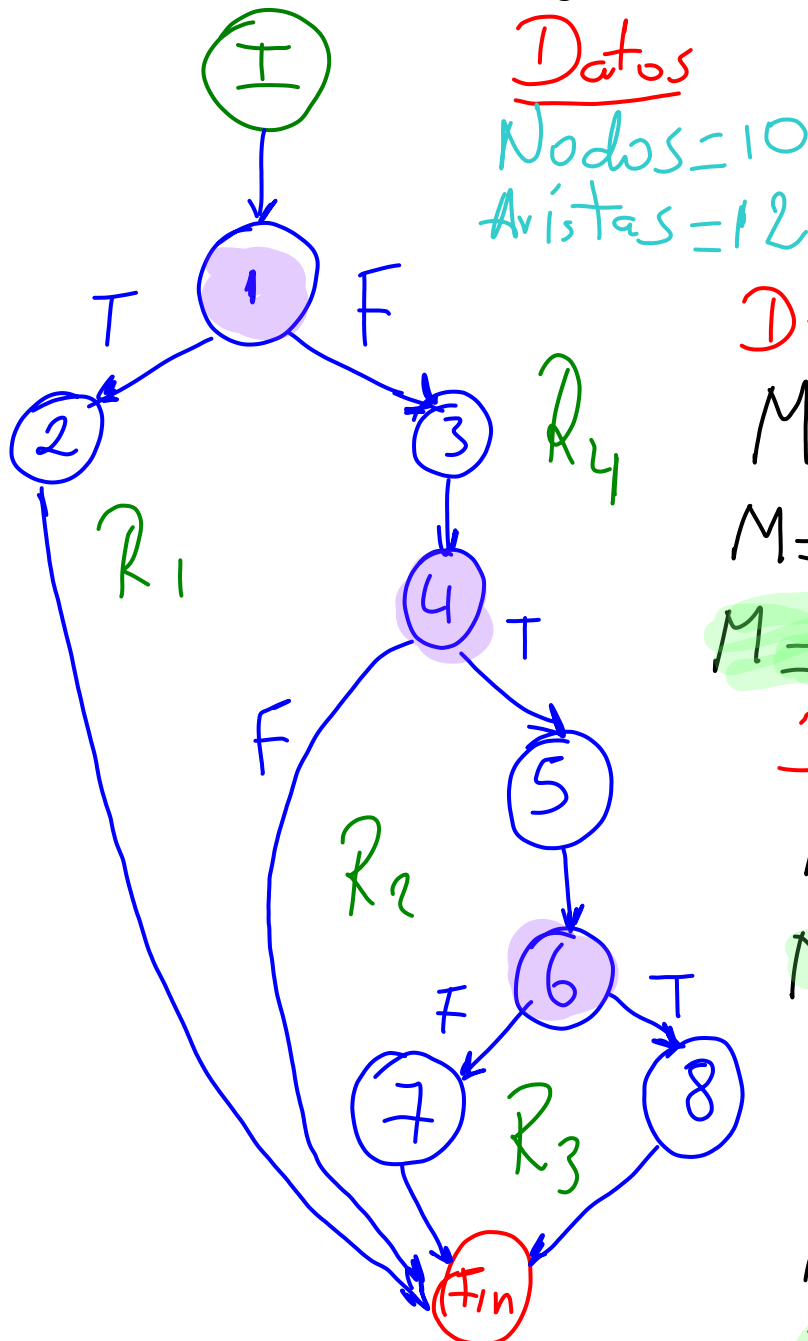
Fin

8

Fin

UNIVERSIDAD DE GUAYAQUIL
 FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
 CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.



Definición 1

$$M = E - N + 2P$$

$$M = 12 - 10 + 2(1)$$

$$M = 4$$

Definición 2

$$M = \# R$$

$$M = 4$$

Definición 3

$$M = NP + 1$$

$$M = 3 + 1$$

$$M = 4$$

Conclusión:

El método tiene una complejidad ciclomática de 4 ($M = 4$):

- Es mantenible
- Es fácil de probar
- Tiene una estructura lógica clara.

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

ANÁLISIS DU – CADENA

Para realizar este análisis se utilizó el método **eliminarMedico()**

variable opcion

define

```
int opcion =  
    JOptionPane.showConfirmDialog(  
        null,  
        "¿Desea eliminar este médico?",  
        "Confirmar eliminación",  
        JOptionPane.YES_NO_OPTION  
    );
```

```
if (opcion == JOptionPane.YES_OPTION) {
```

uso

```
    modelo.setIdMedico(  
        idMedicoSeleccionado  
    );
```

$DU(opcion) = \sim du \sim$

No existe anomalía

variable resultado

define

```
boolean resultado =  
    modelo.eliminarMedico();
```

```
if (resultado) {
```

uso

```
    JOptionPane.showMessageDialog(  
        null,  
        "Médico eliminado correctamente"  
    );
```

```
    limpiarCampos();
```

```
    listarMedicos();
```

```
    idMedicoSeleccionado = 0;
```

```
} else {
```

```
    JOptionPane.showMessageDialog(  
        null,  
        "Error al eliminar médico"  
    );
```

```
}
```

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE INGENIERÍA EN SOFTWARE

Docente: Ing. Verónica Mendoza Morán MSc.

$DU(\text{resultado}) = \sim du \sim$
No existe anomalía

En el análisis DU-Cadena realizado sobre las variables **opcion** y **resultado** del método eliminarMedico(), no se encontraron anomalías de flujo de datos, ya que ambas variables son correctamente definidas antes de ser utilizadas dentro del programa.